



STM32C5 Demo workshop:

STM32CubeMX2 new features

Pape Libasse SOW

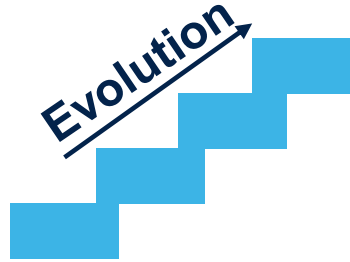
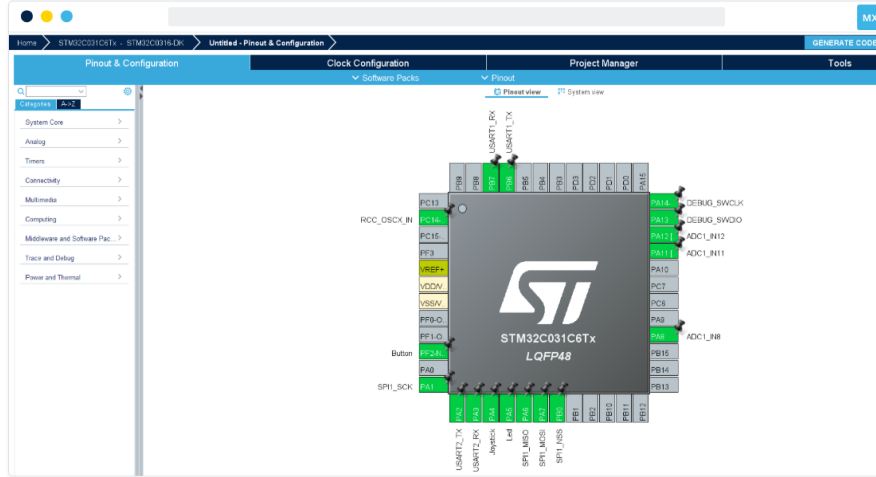


Agenda

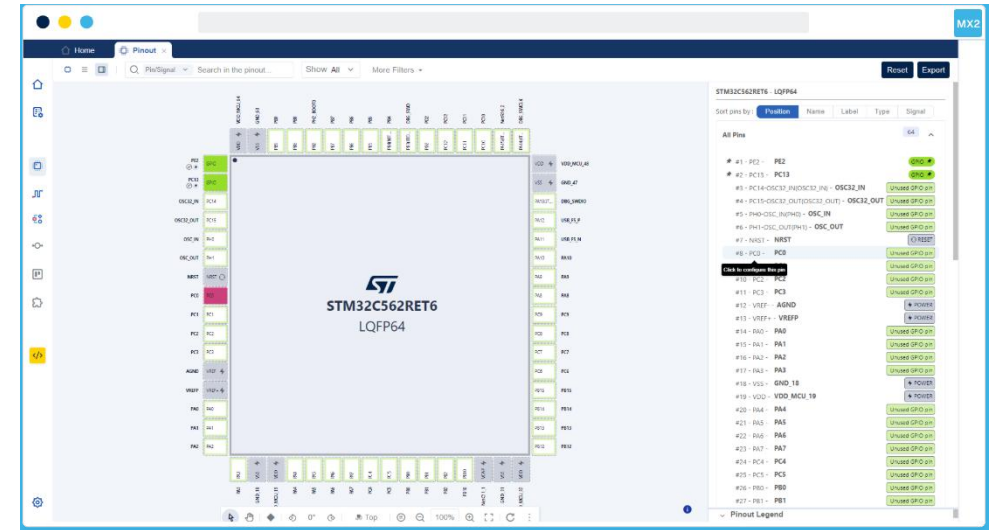
- 1 What is STM32CubeMX2 ?
- 2 STM32CubeMX2 GUI
- 3 STM32CubeMX2 features
- 4 Use case : *code preview usage*

What is STM32CubeMX2?

STM32CubeMX



STM32CubeMX2



- ✓ Based on solid foundation of STM32CubeMX
- ✓ Enhancements
 - ➔ Code preview, clock path, pinout table view, etc.
- ✓ Introduced for new STM32 series (from STM32C5)
 - ➔ Work with MCU that use HAL2 software layer



Increase design efficiency
&
User experience

STM32CubeMX2: GUI and Features





STM32CubeMX2 GUI: Home

Menu bar

Left bar

Main Features:



Start a new project (1)

From an MCU
From a board



Open an existing project (2)

From the file system
From the list of current/ recent projects



Access useful links and
documentation (3)

Menu bar

Left bar

1 Back to Work? Let's Create Something New.

2

3

MCU Create project

Board Create project

Project Open from file system

Last Opened project - 1 minute ago

MyTestApp

STM32C562RET6 Mounted MCU

Open Project

Recent projects

Project Name	Path	Size	Last opened	Last modified	Last modified with
MyTestApp	c:\Users\isowpaper\STM32C5_Demo_Workshop\ProjectMyTestApp.ioc2	59.82 kB	1 minute ago	1 minute ago	MX2 v1.0.0

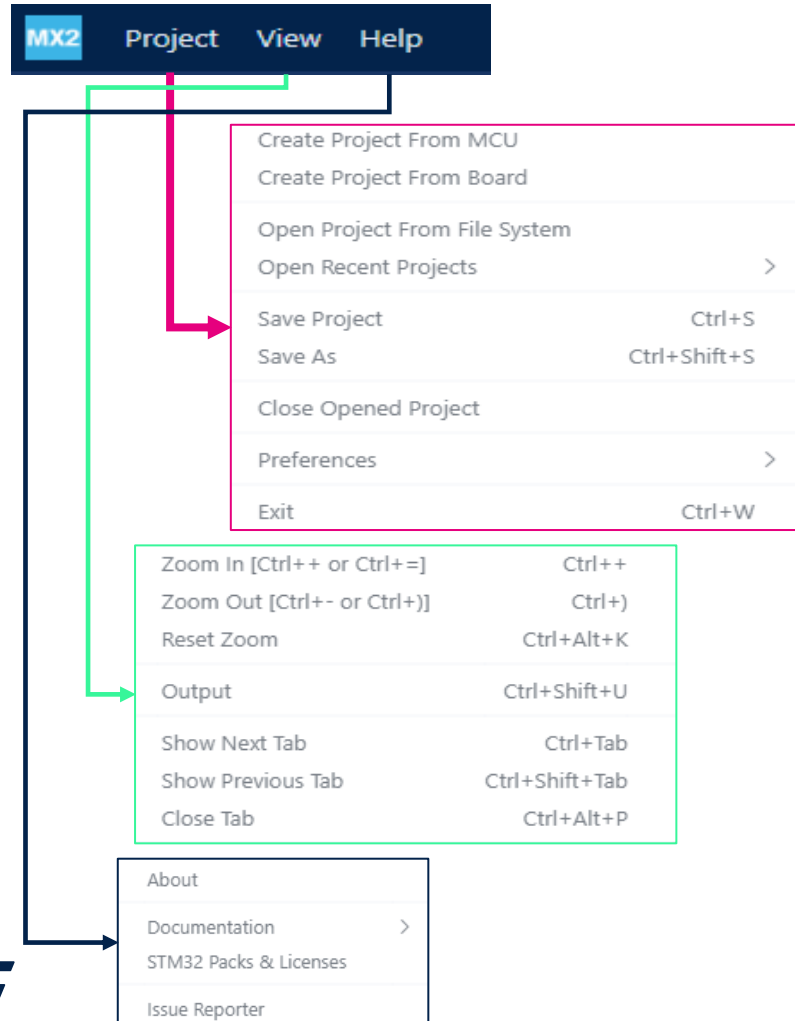
Search project

- STM32 MCU Developer Zone
- STM32CubeMX2 Documentation
- STM32 Embedded Software Documentation
- STM32 MCU Community
- STM32 MCU Documentation
- MCU Wiki page
- STM32 Example browser
- STM32 Package Creator
- STM32 Packs & Licenses

STM32CubeMX2 GUI: Menu & Left Bars

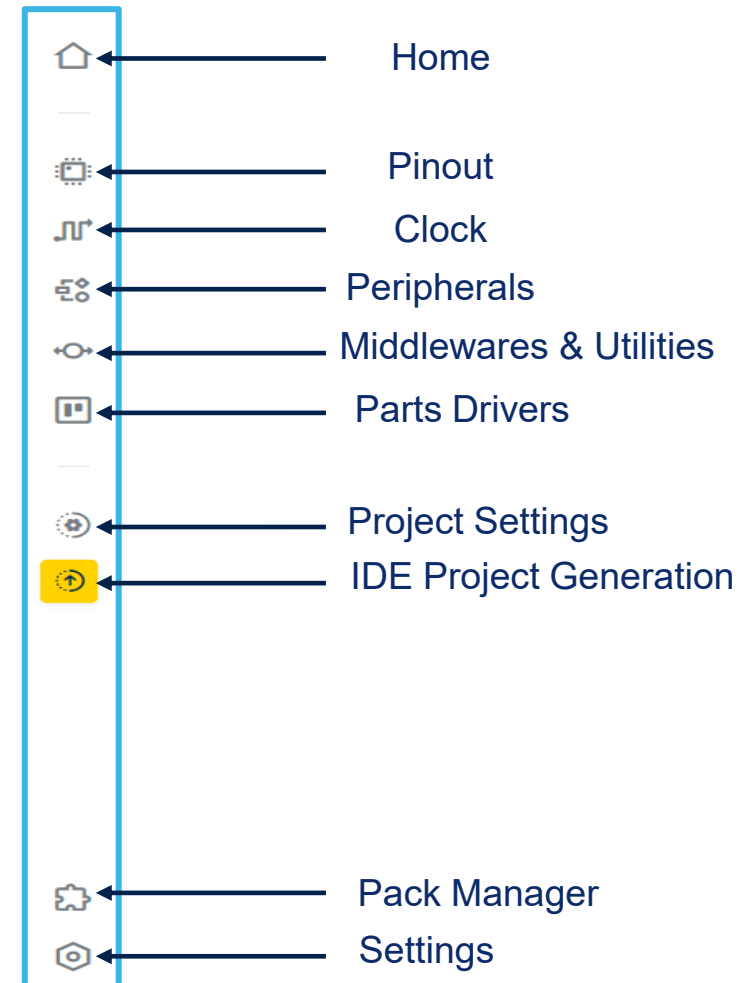
Menu bar

It contains 3 main menus for managing projects and views



Left bar

It contains access icons for different windows and panels



STM32CubeMX2 features: **Pinout, Clock, Code preview**



STM32CubeMX2 features: **Pinout**





Pinout Interface: Search & Filter options

The screenshot shows the Pinout interface for the STM32C562RET6 LQFP64. The interface is divided into several sections:

- Search options (pink dashed box):** Includes a search bar with a dropdown menu showing "All", "Pin", "Signal", "Label", and "Position". A pink arrow points to the "Pin" option.
- Filter options (green dashed box):** Includes filters for "Show", "Type", "I/O structure", "Pin function", "Pin type", "I/O powered by", "Voltage Tolerance", and "Pin Options". A green arrow points to the "Filter options" label.
- Legend (blue dashed box):** Includes a legend for "Pin" with options: "All", "Signal", "Label", and "Position".

Search options

Filter options

- Voltage Tolerance
FT = 5V tolerant I/O
RST = reset pin
TT = 3.6 V tolerant I/O

- Pin Options
a = pin with switch analog function
f = I2C Fm+ capable I/O
t = tamper I/O
u = pin with USB function



The **filters** can be applied based on:

- Pinout legend
- Pin function, Pin type, Source of power
- Voltage Tolerance & Pin options





Pinout Interface: Graphic view

Main Features:

 Configure and manage the MCU's pin assignments and properties (1)

- Assign pin to a signal
- Add/ Remove Pin label
- Access to configuration
- Exclude pin
- Reset pin state

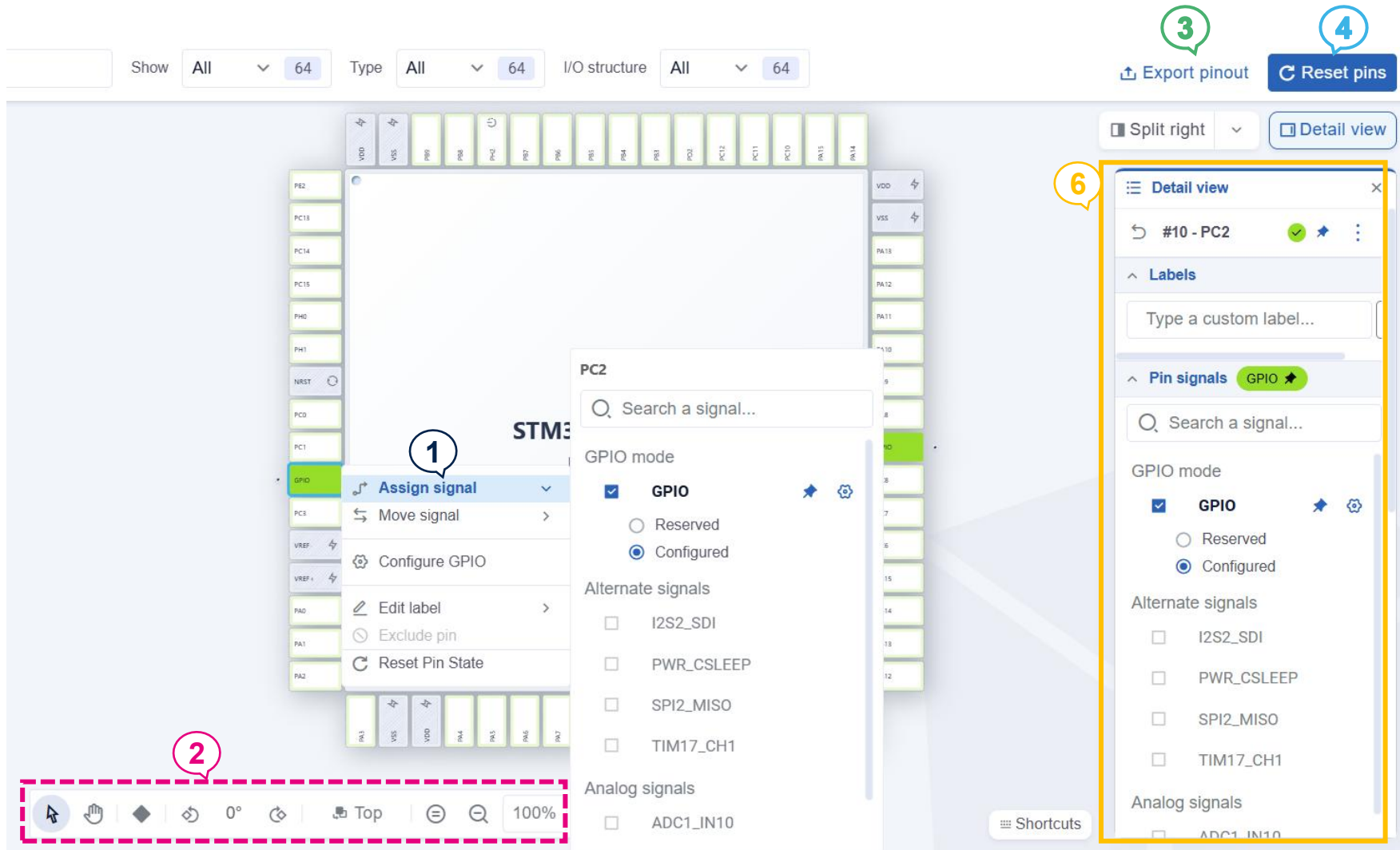
 Toolbar (2)

- Flip package view
- Move/Edit
- Zoom in/out

 Export pinout (4)

 Reset pinout (4)

 Detail view (6)



The screenshot displays the Pinout Interface Graphic view for an STM32 microcontroller. The interface includes a top navigation bar with filters for 'Show' (All), 'Type' (All), and 'I/O structure' (All), all set to '64'. On the right, there are buttons for 'Export pinout' (3) and 'Reset pins' (4). The main area shows a pinout diagram with a context menu open for pin PC2, highlighting the 'Assign signal' option (1). A toolbar at the bottom (2) contains icons for navigation and zoom. On the right, a 'Detail view' panel (6) shows the configuration for pin #10 - PC2, including labels, pin signals (GPIO), and alternate signals.



Pinout interface: Details view

The screenshot displays the STM32CubeMX Pinout interface for the STM32C562RET6 LQFP64 package. The central chip is surrounded by its pins, with PA10 highlighted in blue. To the right, the 'Detail view' panel is open, showing the following information:

- GPIO mode:** ☐ GPIO
- Alternate signals:** ☐ LPUART1_RX, ☐ TIM1_CH3, ☐ USART1_RX
- Pin characteristics:** I/O: Input/output pin, FT: 5V tolerant I/O, VDD: Supplied by VDD
- Pin state:** Pin is not used
- Pinout legend:** Unassigned pins (Not configurable pin, Configurable pin), Assigned pins (Reserved pin, Configured pin), Pin states (Pinned signal(s), Excluded pin (not used), Configuration warning, Configuration error)

Annotations on the right side of the image point to specific sections of the 'Detail view' panel:

- Alternate function:** Points to the 'Alternate signals' section.
- Pin Characteristics:** Points to the 'Pin characteristics' section.
- Pin state:** Points to the 'Pin state' section.
- Pinout legend:** Points to the 'Pinout legend' section.



Pinout interface: Table view

- Search & filter options (1)
- Switch button (2)
- Pin assignment (3)
- Add/remove Labels (4)
- Configure, reset pin state, move signal, exclude (5)

The screenshot shows the Pinout interface in Table view. The interface includes a search bar at the top with a magnifying glass icon and the text "Search for any pin name...". Below the search bar are filters for "Show", "All", "64", "Type", "All", "64", and "I/O structure", "All", "64". A "Graphic view" button is highlighted with a pink box and a "Table view" button is next to it. The main table has columns: Pos., Pin, Status, Signals, Labels, and Actions. The row for pin #15 (PA1) is selected. The "Signals" column for pin #15 shows "GPIO" with a star icon. The "Labels" column for pin #15 shows "Add label...". The "Actions" column for pin #15 shows a gear icon and a star icon. A "Detail view" panel on the right shows the configuration for pin #15 - PA1, including "GPIO mode" (Configured) and "Alternate signals" (LPTIM1_IN1, TIM2_CH2, TIM5_CH2, TIM8_BKIN, TIM15_CH1N, UART4_RX).

Pos.	Pin	Status	Signals	Labels	Actions
#7	NRST				
#14	PA0	Reserved	I2S2_WS		
#15	PA1	Configured	GPIO	Add label...	
#16	PA2				
#17	PA3				
#20	PA4				
#21	PA5				
#22	PA6				
#23	PA7				
#41	PA8				
#42	PA9				
#43	PA10				
#44	PA11				

Selected: #15 - PA1

STM32CubeMX2 features: **Clock**

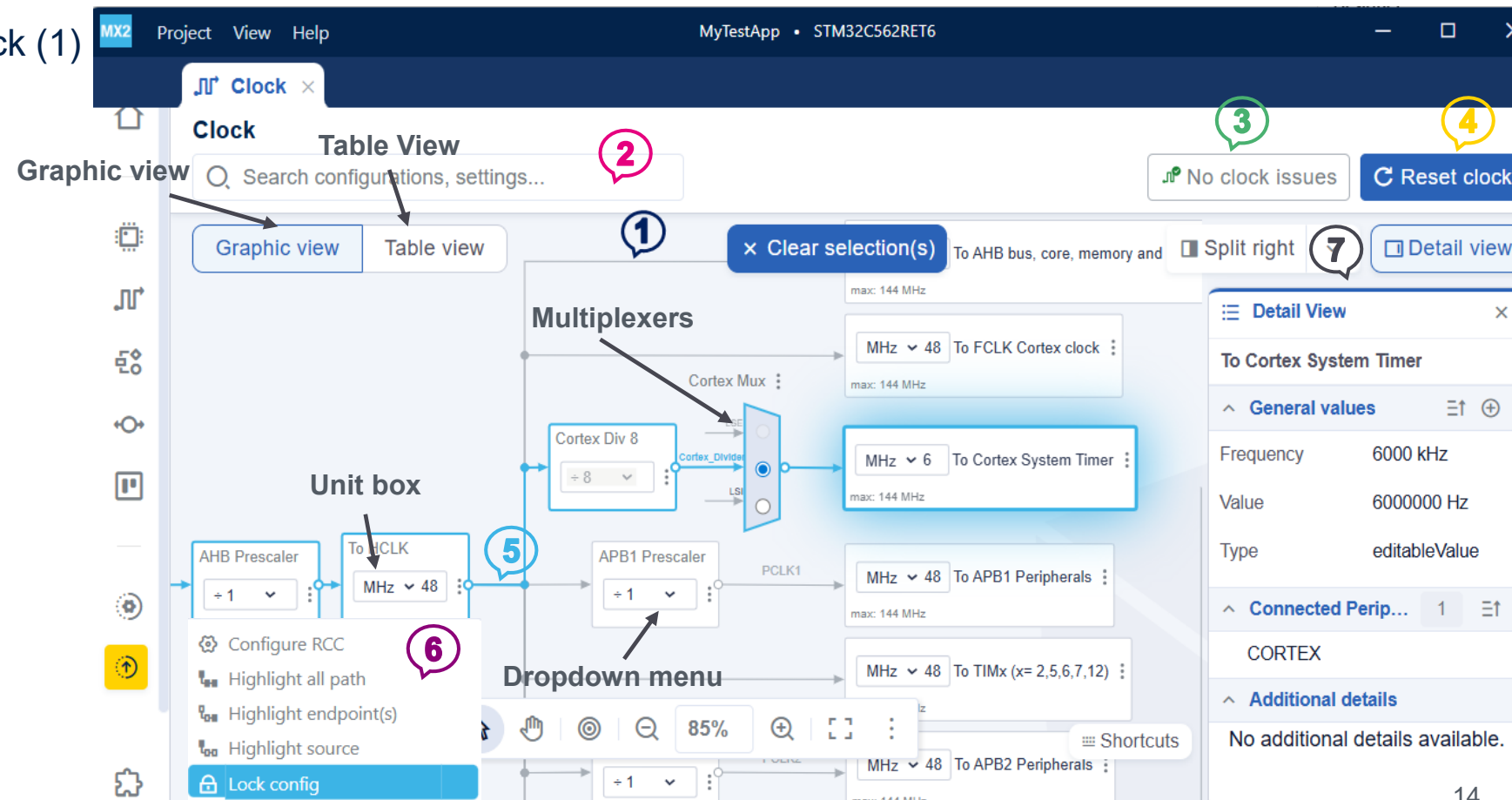




Clock : Graphic View

Main Features:

-  Configure and manage the MCU's clock (1)
-  Search clock (2)
-  Resolve clock issues (3)
-  Reset clock (4)
-  Highlight path (5)
-  Configuration RCC, Lock/ Unlock (6)
-  Toggle details view (7)





Clock: Table View

Main Features:

-  Configure/ Manage the MCU's clock (1)
-  Search clock (2)
-  Resolve clock issues (3)
-  Reset clock (4)
-  Lock/ Unlock configuration (5)
-  Toggle details view (6)

MX2 Project View Help MyTestApp • STM32C562RET6

Clock

Graphic view Table view

Columns Export data

Id	Type	Value	Range	Unit	Output Frequency	Actions
Search						
LSE OSC	variableSource	32.768	min: 1.0 kHz max: 1000.0 kHz	kHz	32768	🔒
LSI RC	variableSource	32	min: 31.4 kHz max: 32.6 kHz	kHz	32000	🔒
HSE OSC	variableSource	8	min: 4.0 MHz max: 50.0 MHz	MHz	8000000	🔒
HSI	fixedSource	144		MHz	144000000	🔒
RTC Div	divider	1			8000000	🔒
RTC Mux	multiplexor	LSI			32000	🔒
To RTC	editableValue	32	max: 1000.0 kHz	kHz	32000	🔒

Selected: 1 line(s) Rows per page: 25 1-25 of 80

Dropdown menu

Text box

1

2

3

4

5

6

No clock issues

Reset clock

Split right

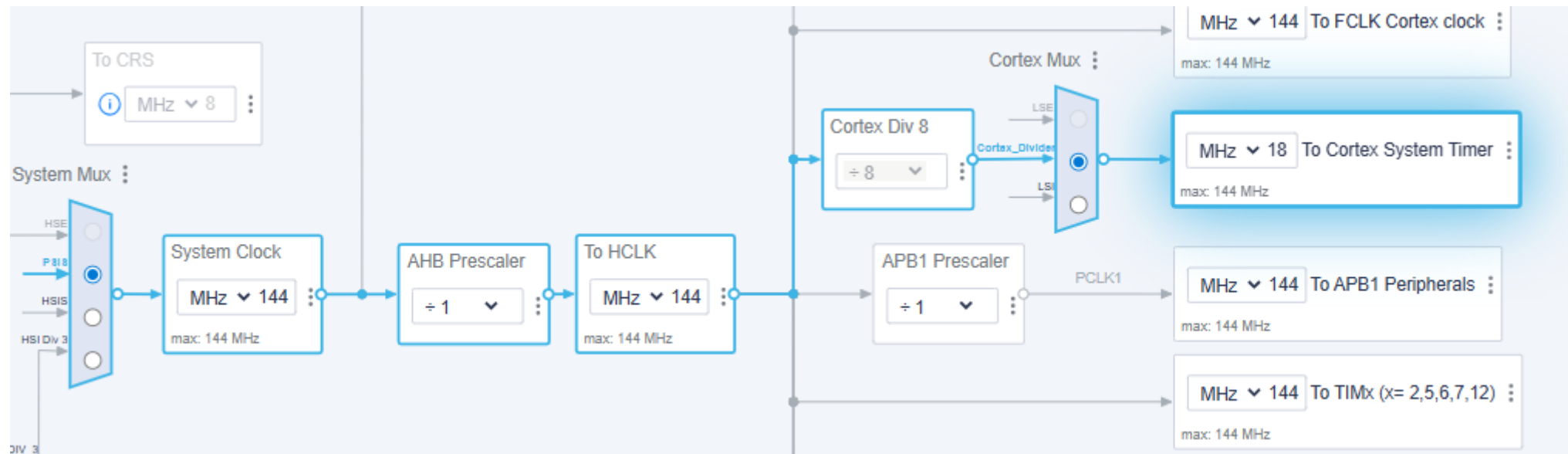
Detail view





Clock: *path, source, endpoints*

- What is the clock source of a specific peripheral ?
- What are the endpoints of a clock source ?
- Go to the RCC configuration



Use case : Code Preview



STM32CubeMX2 features: *Code preview*

1. File selection

2. Show change(s)

3. Show change(s)

4. Settings Menu

Dark Mode

Applicable License

Copy content to clipboard

The screenshot displays the STM32CubeMX2 Code Preview window. On the left, the configuration tree shows 'DMA' selected under 'ADC1_IN0'. The 'DMA request' is set to 'GLOBAL' and 'LPDMA1_CH0'. The 'Show change(s)' button is highlighted with a pink box and a pink arrow labeled '2. Show change(s)'. The 'Refresh' button is highlighted with a green box and a green arrow labeled '3. Show change(s)'. The 'Settings Menu' is highlighted with a yellow box and a yellow arrow labeled '4. Settings Menu'. The 'Dark Mode' toggle is highlighted with a blue box and a blue arrow labeled '1. File selection'. The 'Applicable License' and 'Copy content to clipboard' buttons are also visible in the settings menu.

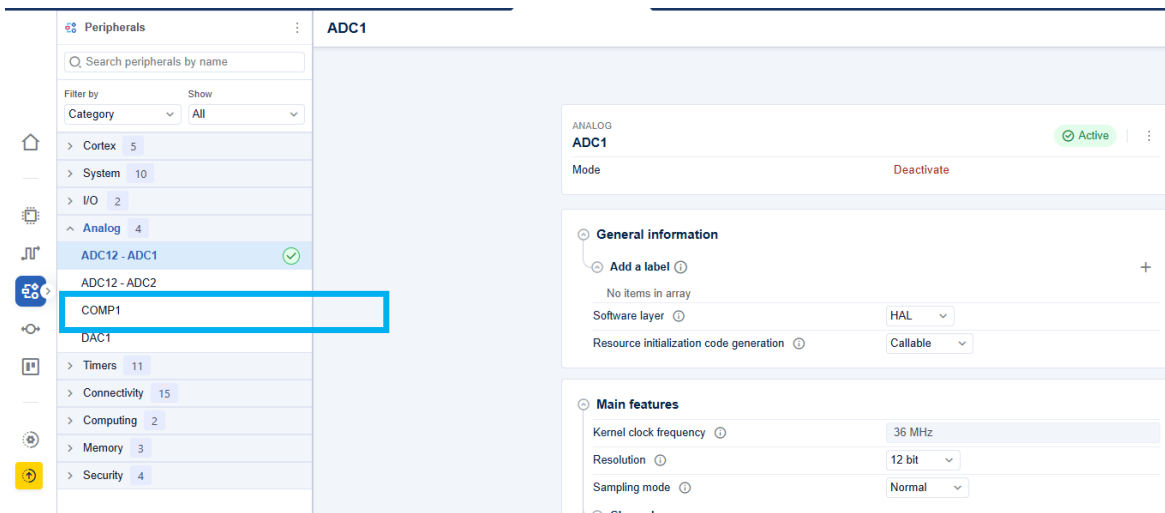
```
84 [GPIO Pin] -----> [Signal Name]
85
86 PA0 -----> ADC1_IN0
87
88 /**
89  * Configure DMA
90  */
91
92 /* Configure DMA */
93 if (HAL_DMA_Init(&hLPDMA1_CH0, HAL_LPDM1_CH0) != HAL_OK)
94 {
95     return NULL;
96 }
97
98 HAL_RCC_LPDM1_EnableClock();
99
100 hal_dma_direct_xfer_config_t xfer_cfg_adc1_dma;
101 xfer_cfg_adc1_dma.request = HAL_LPDM1_REQUEST_ADC1;
102 xfer_cfg_adc1_dma.direction = HAL_DMA_DIRECTION_PERIPH_TO_MEMORY;
103 xfer_cfg_adc1_dma.src_inc = HAL_DMA_SRC_ADDR_FIXED;
104 xfer_cfg_adc1_dma.dest_inc = HAL_DMA_DEST_ADDR_INCREMENTED;
105 xfer_cfg_adc1_dma.src_data_width = HAL_DMA_SRC_DATA_WIDTH_WORD;
106 xfer_cfg_adc1_dma.dest_data_width = HAL_DMA_DEST_DATA_WIDTH_WORD;
107 xfer_cfg_adc1_dma.priority = HAL_DMA_PRIORITY_LOW_WEIGHT_LOW;
108
109 if (HAL_DMA_SetConfigPeriphDirectXfer(&hLPDMA1_CH0, &xfer_cfg_adc1_dma)
110 {
111     return NULL;
112 }
113
114 /* Enable the interruption for LPDM1_CH0 */
115 HAL_CORTEX_NVIC_SetPriority(LPDM1_CH0_IRQn, HAL_CORTEX_NVIC_PREEMP_PRIOR
116 HAL_CORTEX_NVIC_EnableIRQ(LPDM1_CH0_IRQn);
117
118 /* Link DMA handle to ADC handle */
119 HAL_ADC_REG_SetDMA(&hADC1, &hLPDMA1_CH0);
```


Code preview use case

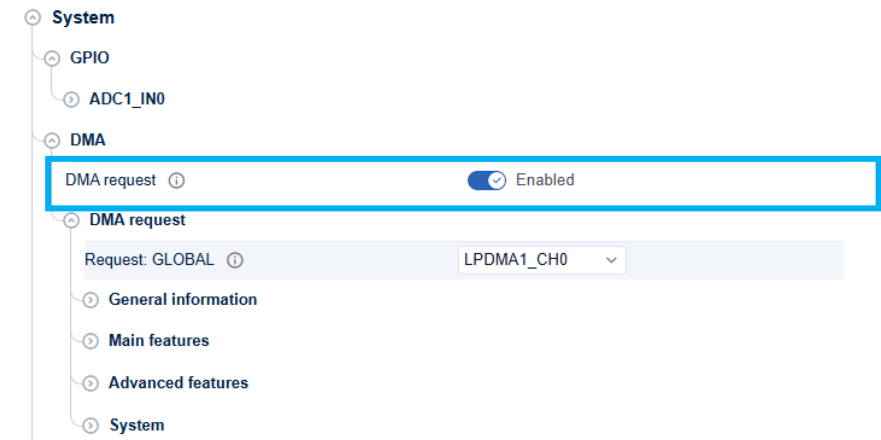
How can I associate my ADC with a DMA in my project without regenerating code?



1 Go to peripheral section and click on “ADC12 – ADC1” section



2 Go to system section and enabled DMA request



Code preview use case

3 Click on “Code preview” section

ADC1

Open STM32 documentation

Split right Quick menu </> Code preview

ADC1

General information

Add a label

Main features

Channels

ADC groups conversions

Advanced features

System

GPIO

DMA

DMA request

Request: GLOBAL LPDMA1_CH0

General information

Code Preview

File mx_adc1.c

Show change(s) Refresh

```
95 return NULL;
96 }
97
98 HAL_RCC_LPDMA1_EnableClock();
99
100 hal_dma_direct_xfer_config_t xfer_cfg_adc1_dma;
101 xfer_cfg_adc1_dma.request = HAL_LPDMA1_REQUEST;
102 xfer_cfg_adc1_dma.direction = HAL_DMA_DIRECTION_I;
103 xfer_cfg_adc1_dma.src_inc = HAL_DMA_SRC_ADDR_F;
104 xfer_cfg_adc1_dma.dest_inc = HAL_DMA_DEST_ADDR_F;
105 xfer_cfg_adc1_dma.src_data_width = HAL_DMA_SRC_DATA_W;
106 xfer_cfg_adc1_dma.dest_data_width = HAL_DMA_DEST_DATA_W;
107 xfer_cfg_adc1_dma.priority = HAL_DMA_PRIORITY_L;
108
109 if (HAL_DMA_SetConfigPeriphDirectXfer(&hLPDMA1_CH0, &xfer_cfg_adc1_dma))
110 {
111     return NULL;
112 }
113
114 /* Enable the interruption for LPDMA1_CH0 */
115 HAL_CORTEX_NVIC_SetPriority(LPDMA1_CH0_IRQn, HAL_CORTEX_NVIC_PRIORITY_1);
116 HAL_CORTEX_NVIC_EnableIRQ(LPDMA1_CH0_IRQn);
117
118 /* Link DMA handle to ADC handle */
119 HAL_ADC_REG_SetDMA(&hADC1, &hLPDMA1_CH0);
120
121 return &hADC1;
122 }
```

Code preview use case

- 4 Navigate through “mx_adc1.c” and copy the “initialization” section of the DMA section in the project

ADC12.ADC1

File **mx_adc1.c** → mx_adc1.c

Show change(s) Refresh

```
97  return NULL;
98  }
99
100 HAL_RCC_LPDMA1_EnableClock();
101
102 hal_dma_direct_xfer_config_t xfer_cfg_adc1_dma;
103 xfer_cfg_adc1_dma.request      = HAL_LPDMA1_REQUEST_ADC1;
104 xfer_cfg_adc1_dma.direction    = HAL_DMA_DIRECTION_PERIPH_TO_MEMORY;
105 xfer_cfg_adc1_dma.src_inc      = HAL_DMA_SRC_ADDR_FIXED;
106 xfer_cfg_adc1_dma.dest_inc     = HAL_DMA_DEST_ADDR_INCREMENTED;
107 xfer_cfg_adc1_dma.src_data_width = HAL_DMA_SRC_DATA_WIDTH_HALFWORD;
108 xfer_cfg_adc1_dma.dest_data_width = HAL_DMA_DEST_DATA_WIDTH_HALFWORD;
109 xfer_cfg_adc1_dma.priority     = HAL_DMA_PRIORITY_LOW_WEIGHT_LOW;
110
111 if (HAL_DMA_SetConfigPeriphLinkedListCircularXfer(&hLPDMA1_CH0, &DMA_Node_ADC1))
112 {
113     return NULL;
114 }
115
116 /* Enable the interruption for LPDMA1_CH0 */
117 HAL_CORTEX_NVIC_SetPriority(LPDMA1_CH0_IRQn, HAL_CORTEX_NVIC_PREEMP_PRIORITY_0);
118 HAL_CORTEX_NVIC_EnableIRQ(LPDMA1_CH0_IRQn);
119
120 /* Link DMA handle to ADC handle */
121 HAL_ADC_REG_SetDMA(&hADC1, &hLPDMA1_CH0);
122
123 return &hADC1;
124 }
125
126 void mx_adc1_deinit(void)
127 {
128     (void)HAL_ADC_DeInit(&hADC1);
129
130     /* De-initialize all GPIO pins associated with ADC1 */
131     HAL_GPIO_DeInit(HAL_GPIOA, HAL_GPIO_PIN_0);
132
133     /* De-initialize the DMA channel */
134     HAL_DMA_DeInit(&hLPDMA1_CH0);
135 }
```

VSCode Project

mx_adc1.c ×

generated > hal > C mx_adc1.c > mx_adc1_deinit

```
35  {
36
37  }
38
39  HAL_GPIO_Init(gpiox: HAL_GPIOA, pins: HAL_GPIO_PIN_0, p_config: &gpio_config);
40
41  /* Configure DMA */
42  if (HAL_DMA_Init(hdma: &hLPDMA1_CH0, instance: HAL_LPDMA1_CH0) != HAL_OK)
43  {
44      return NULL;
45  }
46
47  HAL_RCC_LPDMA1_EnableClock();
48
49  hal_dma_direct_xfer_config_t xfer_cfg_adc1_dma;
50  xfer_cfg_adc1_dma.request      = HAL_LPDMA1_REQUEST_ADC1;
51  xfer_cfg_adc1_dma.direction    = HAL_DMA_DIRECTION_PERIPH_TO_MEMORY;
52  xfer_cfg_adc1_dma.src_inc      = HAL_DMA_SRC_ADDR_FIXED;
53  xfer_cfg_adc1_dma.dest_inc     = HAL_DMA_DEST_ADDR_INCREMENTED;
54  xfer_cfg_adc1_dma.src_data_width = HAL_DMA_SRC_DATA_WIDTH_HALFWORD;
55  xfer_cfg_adc1_dma.dest_data_width = HAL_DMA_DEST_DATA_WIDTH_HALFWORD;
56  xfer_cfg_adc1_dma.priority     = HAL_DMA_PRIORITY_LOW_WEIGHT_LOW;
57
58  if (HAL_DMA_SetConfigPeriphLinkedListCircularXfer(hdma: &hLPDMA1_CH0, p_node: &DMA_Node_ADC1, p_node_config: &xfer_cfg_adc1_dma) != HAL_OK)
59  {
60      return NULL;
61  }
62
63  /* Enable the interruption for LPDMA1_CH0 */
64  HAL_CORTEX_NVIC_SetPriority(irqn: LPDMA1_CH0_IRQn, preemp_prio: HAL_CORTEX_NVIC_PREEMP_PRIORITY_0, sub_prio: HAL_CORTEX_NVIC_SUB_PRIORITY_0);
65  HAL_CORTEX_NVIC_EnableIRQ(irqn: LPDMA1_CH0_IRQn);
66
67  /* Link DMA handle to ADC handle */
68  HAL_ADC_REG_SetDMA(hadc: &hADC1, hdma: &hLPDMA1_CH0);
69
70 }
```

Code preview use case

- 5 Navigate through “mx_adc1.c” and copy the “de-initialization” section of the DMA section in the project

ADC12.ADC1
Code Preview

File **mx_adc1.c**   mx_adc1.c

☐ Show change(s)

```
116  /* Enable the interruption for LPDMA1_CH0 */
117  HAL_CORTEX_NVIC_SetPriority(LPDMA1_CH0_IRQn, HAL_CORTEX_NVIC_PREEMP_PRIOR
118  HAL_CORTEX_NVIC_EnableIRQ(LPDMA1_CH0_IRQn);
119
120  /* Link DMA handle to ADC handle */
121  HAL_ADC_REG_SetDMA(&hADC1, &hLPDMA1_CH0);
122
123  return &hADC1;
124 }
125
126 void mx_adc1_deinit(void)
127 {
128     (void)HAL_ADC_DeInit(&hADC1);
129
130     /* De-initialize all GPIO pins associated with ADC1 */
131     HAL_GPIO_DeInit(HAL_GPIOA, HAL_GPIO_PIN_0);
132
133     /* De-initialize the DMA channel */
134     HAL_DMA_DeInit(&hLPDMA1_CH0);
135
136     /* Disable the interruption for DMA */
137     HAL_CORTEX_NVIC_DisableIRQ(LPDMA1_CH0_IRQn);
138 }
139
```

VSCode Project

mx_adc1.c x

generated > hal > C mx_adc1.c > mx_adc1_deinit

```
132
133 void mx_adc1_deinit(void)
134 {
135     (void)HAL_ADC_DeInit(&hADC1);
136
137     /* De-initialize all GPIO pins associated with ADC1 */
138     HAL_GPIO_DeInit(gpiox: HAL_GPIOA, pins: HAL_GPIO_PIN_0);
139
140     /* De-initialize the DMA channel */
141     HAL_DMA_DeInit(hdma: &hLPDMA1_CH0);
142
143     /* Disable the interruption for DMA */
144     HAL_CORTEX_NVIC_DisableIRQ(irqn: LPDMA1_CH0_IRQn);
145 }
146
```

Code preview use case

- 5 Navigate through “mx_adc1.h” and copy the related configuration section in the project

ADC12.ADC1
Code Preview

File mx_adc1.h ↗ mx_adc1.h

Show change(s) Refresh

```
32
33 /* Exported macros -----
34 /* Exported variables -----
35 /* Exported functions -----
36 /*-----
37 /* Exported functions for ADC in HAL layer */
38 /*-----
39 /**
40  * @brief mx_adc1 init function
41  * This function configures the hardware resources used in this example
42  * @retval pointer to handle or NULL in case of failure
43  */
44 hal_adc_handle_t *mx_adc1_init(void);
45
46 /**
47  * @brief De-initialize adc1 instance and return it.
48  */
49 void mx_adc1_deinit(void);
50
51 /**
52  * @brief Get the ADC1 object.
53  * @retval Pointer on the ADC1 Handle
54  */
55 hal_adc_handle_t *mx_adc1_gethandle(void);
56
57 /*-----
58 /* LPDMA1 channel0 global interrupt
59 /*-----
60
61 void LPDMA1_CH0_IRQHandler(void);
62
63 #ifdef __cplusplus
64 }
65 #endif /* __cplusplus */
66
67 #endif /* MX_ADC1_H */
```

VSCoDe Project

h mx_adc1.h ×

generated > hal > h mx_adc1.h > ...

```
24 extern "C" {
33 /* Exported macros -----*/
42 * @retval pointer to handle or NULL in case of failure
43 */
44 hal_adc_handle_t *mx_adc1_init(void);
45
46 /**
47  * @brief De-initialize adc1 instance and return it.
48  */
49 void mx_adc1_deinit(void);
50
51 /**
52  * @brief Get the ADC1 object.
53  * @retval Pointer on the ADC1 Handle
54  */
55 hal_adc_handle_t *mx_adc1_gethandle(void);
56
57 /*-----
58 /* LPDMA1 channel0 global interrupt
59 /*-----
60
61 void LPDMA1_CH0_IRQHandler(void);
62
```

Code preview use case

- 6 Set the “USE_HAL_DMA” to 1U in the file “stm32C5xx_hal_conf.h”

```
h stm32c5xx_hal_conf.h X
generated > hal > h stm32c5xx_hal_conf.h
23  extern "C" {
98
99  /** @defgroup HAL_ADC_Config HAL ADC Configuration
100  * @{
101  */
102  /* ##### HAL_ADC Config ##### */
103  #define USE_HAL_ADC_MODULE                1U
104  #define USE_HAL_ADC_CLK_ENABLE_MODEL      HAL_CLK_ENABLE_NO
105  #define USE_HAL_ADC_REGISTER_CALLBACKS    0U
106  #define USE_HAL_ADC_USER_DATA             0U
107  #define USE_HAL_ADC_GET_LAST_ERRORS       0U
108  #define USE_HAL_ADC_DMA                   1U
109  /**
110  * @}
```


Code preview use case

- 7 Set the “USE_HAL_DMA_LINKEDLIST” to 1U in the file “stm32C5xx_hal_conf.h”

```
h stm32c5xx_hal_conf.h ×
generated > hal > h stm32c5xx_hal_conf.h
23  extern "C" {
231  /** @defgroup HAL_DMA_Config HAL DMA Configuration
232  *   @{
233  */
234  /* ##### HAL_DMA Config ##### */
235  #define USE_HAL_DMA_MODULE                1U
236  #define USE_HAL_DMA_CLK_ENABLE_MODEL     HAL_CLK_ENABLE_NO
237  #define USE_HAL_DMA_USER_DATA            0U
238  #define USE_HAL_DMA_GET_LAST_ERRORS      0U
239  #define USE_HAL_DMA_LINKEDLIST           1U
```

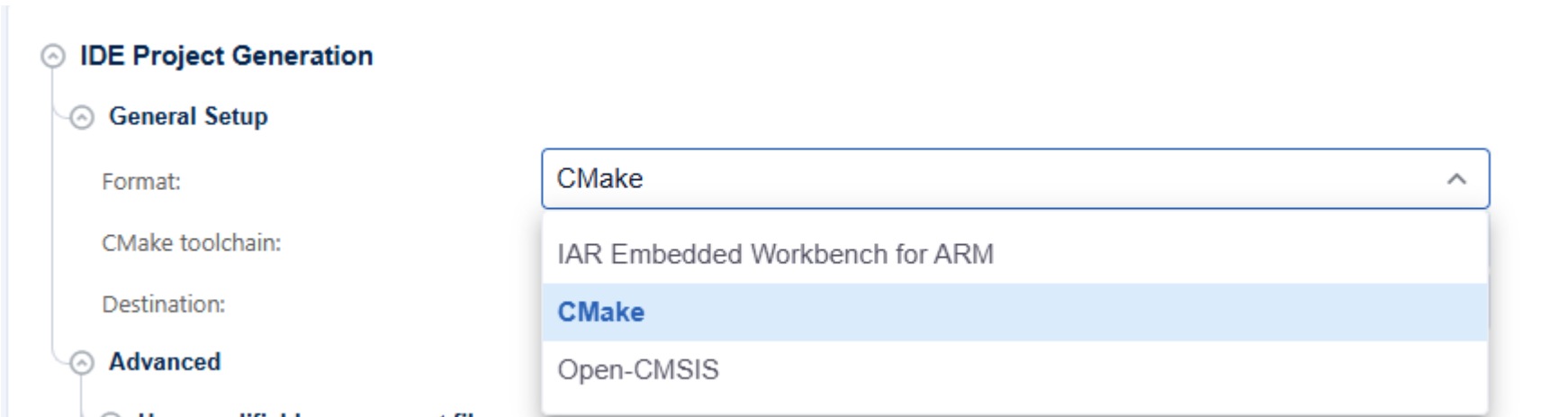
STM32CubeMX2 features: **Project settings**



Project settings: Available formats

STM32CubeMX2 offers the possibility to generate code for these three format

- **IAR Embedded Workbench for ARM**
- **CMake** (required for STM32Cube for VSCode and STM32CubeIDE v2.1.0)
- **Open-CMSIS** (compatible with Keil μ Vision and Keil Studio)



Project settings: Toolchain

STM32CubeMX2 offers dedicated Toolchain depending on the project format

- IAR Embedded Workbench for ARM : **IAR**
- CMake : **GCC**
- Open-CMSIS : **AC6, GCC, IAR**

IAR Embedded Workbench for ARM

IDE Project Generation

General Setup

Format: IAR Embedded Workbench for ARM

EWARM toolchain: IAR

Destination: IAR

CMake

IDE Project Generation

General Setup

Format: CMake

CMake toolchain: GCC

Destination: GCC

Open CMSIS

IDE Project Generation

General Setup

Format: Open-CMSIS

Open-CMSIS toolchain: AC6

Destination: AC6

Advanced

User modifiable component files

GCC

IAR

Project settings: *Automatic Conflict Handling Rules*

STM32CubeMX “**user code**” sections are removed and replaced by the “***Automatic Conflict Handling Rules***” feature

The image shows the STM32CubeMX project settings interface. On the left, a green box highlights the "user code" sections, marked with `/* USER CODE BEGIN X */` and `/* USER CODE END X */`. A green arrow points from this box to the "Automatic Conflict Handling Rules" section in the settings panel on the right. The settings panel includes sections for Generator, Packs, Installation Directories, and Automatic Conflict Handling Rules. The "Automatic Conflict Handling Rules" section is highlighted with a blue dashed border, showing a table with columns for Category and Rule. The "All" category is selected, and the rule is set to "Update files on change". A yellow button labeled "Generate IDE project" is located below the settings panel.

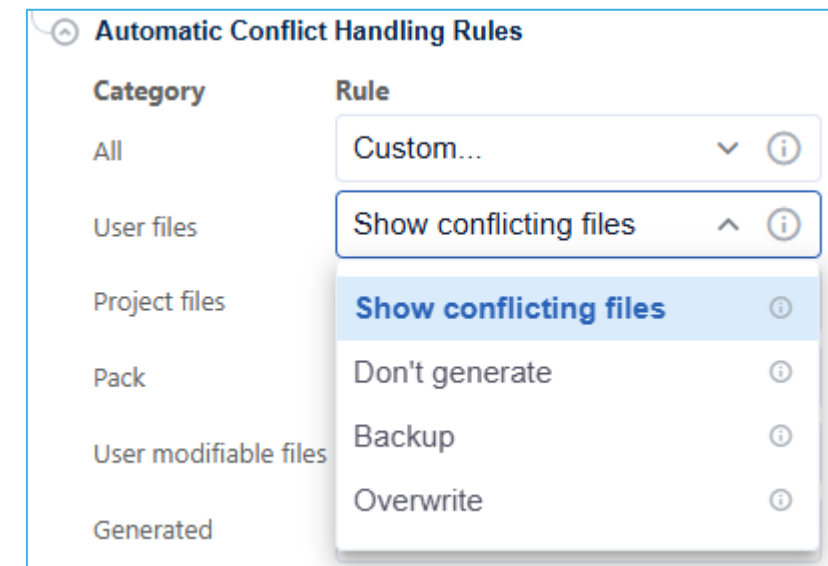
Category	Rule
All	Update files on change

By default, all files are updated on change

Project settings: *Automatic Conflict Handling Rules*

Instead of “**update files on change**” option, the “**custom**” one can be used
→ to handle different types of **conflict resolution**

- **Show conflicting files** : launch the conflict resolution tool to handle each case
- **Don't generate** : Files will be kept unchanged
- **Backup** : Creates backup of existing files (with **_Bckp** suffix) and overwrites them
- **Overwrite** : Overwrites the existing



✓ **STM32CubeMX2 GUI features**

✓ **Project creation**

✓ **Project configuration**

✓ **A code preview use case**

✓ **Project management**

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.

